



Widgets

Specification

iCUE Widget Specification

WIDGET API VERSION 1.4.0

QtWebEngine: 6.9.3

Chromium: 130.0.6723.192

Minimum iCUE: 5.47

This document provides the technical specification for the iCUE Widgets, including the HTML widget structure, manifest schema, and iCUE communication protocol.

Widget File Structure

Required Files

- `index.html` – Main HTML entry point
- `manifest.json` – Widget manifest and metadata
- `icon.svg` or `icon.png` – Widget icon

Directory Structure

```
mywidget/  
├── index.html  
├── manifest.json  
├── translation.json  
├── modules/  
│   └── CitySearch.mjs  
├── scripts/  
│   └── widget.js  
├── styles/  
│   └── style.css  
└── resources/  
    ├── icon.png  
    └── thumbnail.png
```



HTML Structure

Widgets consist of four primary sections:

- **Head** – Metadata, [widget controls](#), widget name (`<title>`), and icon (`<link rel="icon">`)
- **Style** – CSS definitions (inline or external file)
- **Body** – Visual elements and DOM structure
- **Script** – Widget logic and event handlers

Minimal HTML Example

```
<!doctype html>
<html lang="en">
  <head>
    <title>My Widget</title>
    <link rel="icon" type="image/x-icon" href="resources/icon.png" />
  </head>
  <body>
    <!-- Widget content -->
  </body>
</html>
```

Manifest Schema

The `manifest.json` file defines widget metadata, system requirements, and marketplace information. All paths in the manifest are relative to the manifest file location.

Schema Properties

Property	Type	Required	Description
<code>author</code>	<code>string</code>	Yes	Widget author name
<code>id</code>	<code>string</code>	Yes	Unique widget identifier in reverse-DNS format (e.g.,

Property	Type	Required	Description
			<code>com.author.widget</code>)
<code>name</code>	<code>string</code>	Yes	Widget display name for marketplace
<code>description</code>	<code>string</code>	Yes	Widget description for marketplace
<code>version</code>	<code>string</code>	Yes	Widget version in semver format (e.g., <code>1.0.0</code>)
<code>preview_icon</code>	<code>string</code>	Yes	Path to widget icon file (relative to manifest)
<code>min_framework_version</code>	<code>string</code>	Yes	Minimum required iCUE Widget API version (e.g., <code>1.0.0</code>)
<code>os</code>	<code>object[]</code>	Yes	Array of supported operating systems objects. Each object requires a <code>platform</code> property. At the moment only <code>windows</code> is supported.
<code>supported_devices</code>	<code>object[]</code>	Yes	Array of supported device types (see Device Support)
<code>interactive</code>	<code>boolean</code>	No	Enable widget click handling on touch devices (default: <code>false</code>)
<code>required_plugins</code>	<code>string[]</code>	No	Array of required plugins in <code>namespace:Name:version</code> format
<code>modules</code>	<code>string[]</code>	No	Array of JavaScript module paths (see Module Integration)
<code>min_app_version</code>	<code>string</code>	Yes	Minimum recommended iCUE version (e.g., <code>5.47</code>)

Example Manifest

```
{
  "author": "Corsair Team",
  "id": "com.corsair.weatherwidget",
  "name": "Weather Widget",
  "description": "Display current weather and forecast",
  "version": "1.0.0",
  "preview_icon": "resources/icon.png",
  "min_app_version": "5.46",
  "min_framework_version": "1.0.0",
  "os": [
    {
      "platform": "windows"
    }
  ],
  "supported_devices": [
    {
      "type": "dashboard_lcd",
      "features": ["sensor-screen"]
    },
    {
      "type": "pump_lcd"
    },
    {
      "type": "keyboard_lcd"
    }
  ],
  "interactive": true,
  "required_plugins": [
    "widgetbuilder.sensorsdataprotider:Sensors:1.0",
    "widgetbuilder.mediadataprovider:Media:1.0"
  ],
  "modules": [
    "modules/CitySearch.mjs"
  ]
}
```

Device Support

The `supported_devices` array specifies compatible device types and optional feature requirements.

Device Types

Type	Description	Examples
<code>dashboard_lcd</code>	Dashboard displays	XENEON EDGE
<code>keyboard_lcd</code>	Keyboards with integrated LCD screens	VANGUARD 96, VANGUARD PRO 96
<code>pump_lcd</code>	AIO cooler pumps with LCD screens	iCUE LINK XC7/XD5 ELITE LCD

Device Features

Optional `features` array restricts widget to devices with specific capabilities.

Feature	Description
<code>sensor-screen</code>	Device supports sensor data display

Example:

```
"supported_devices": [  
  {  
    "type": "dashboard_lcd",  
    "features": ["sensor-screen"]  
  },  
  {  
    "type": "pump_lcd"  
  }  
]
```

HTML Meta Tags

Meta tags in the `<head>` section control widget presentation in iCUE and define user-configurable properties.

Required Meta Tags

Element	Description
<code><title></code>	Widget name displayed in iCUE in widgetselector. Use <code>tr('Name')</code> for translations
<code><link rel="icon"></code>	Widget icon displayed in iCUE selector. Supported formats: SVG, PNG, ICO

Optional Meta Tags

Meta Tag Name	Description
<code>x-icue-widget-group</code>	Group name in widget selector in iCUE (for example <code>tr('Clock Face'))</code>
<code>x-icue-widget-preview</code>	Preview image path for widget selector (128x56px PNG)

Widget Grouping

Widgets in the same `x-icue-widget-group` will be grouped together in the widget selector and displayed as variants of the same widget. Widgets with identical names are automatically grouped together.

```
<meta name="x-icue-widget-group" content="tr('Clock Face')" />
```

NOTE

The `tr()` function enables [localized group names](#).

Widget Preview

The `x-icue-widget-preview` meta tag specifies a preview image shown in the widget selector before installation. The image should be 128x56 pixels in size and preferably in PNG format. The path to the image is relative to the `index.html` file.

```
<meta name="x-icue-widget-preview" content="resources/preview.png" />
```

Widget Controls

Widget controls are user-configurable parameters defined as `<meta>` tags with `name="x-icue-property"`. Each control becomes a global JavaScript variable in your widget.

```
<meta name="x-icue-property" content="textColor" data-label="Text Color" data-type="color" data-default="'#FFFFFF'" />
```

INFO

For complete documentation on available control types, attributes, and examples, see [Widget Controls](#).

Property Groups

Property groups organize controls into panels in the iCUE. Groups are defined in a JSON array within a `<script>` element with `id="x-icue-groups"`. JavaScript expressions are supported in `title` and `info` fields.

Group Schema

```
<script type="application/json" id="x-icue-groups">
[
  {
    "title": "tr('Settings')",
    "properties": ["option1", "option2"]
    "info": "tr('Optional help text for this group')"
  },
  {
    "title": "tr('Appearance')",
    "properties": ["textColor", "backgroundColor", "backgroundMedia"],
  }
]
</script>
```

Group Properties

Property	Type	Required	Description
<code>title</code>	<code>string</code>	Yes	Group heading displayed in settings panel. Supports JavaScript expressions including <code>tr()</code> for translations
<code>properties</code>	<code>string[]</code>	Yes	Array of meta parameters (<code>content</code> values) to include in this group
<code>info</code>	<code>string</code>	No	Optional help text displayed as tooltip. Supports JavaScript expressions including <code>tr()</code>

Device-Specific Behavior

XENEON EDGE Custom Styles:

On XENEON EDGE devices, property groups containing `textColor`, `accentColor`, or `backgroundColor` automatically display a "Custom Style" toggle switch in the settings panel (second tab only). When disabled, the device's default color scheme is applied instead of custom values.

This behavior is device-specific and only applies to XENEON EDGE.

iCUE Events

iCUE injects script blocks into the widget's HTML page at runtime. These scripts provide:

- **Global variables** for each meta parameter (e.g., `textColor`, `fontSize`)
- `iCUE` global object with utility functions (see [iCUE Global Object](#))
- `iCUE_initialized` flag indicating API readiness
- `device` object with information about the device displaying the widget (see [Device Object](#))
- **Plugin objects** in `window.plugins` namespace [more info about plugins](#)

The injection occurs before the widget's own scripts execute, making all iCUE data immediately available.

Event handlers:

Register callbacks via the global `icueEvents` object:

```
icueEvents = {
  onDataUpdated: onIcueDataUpdated,
  onICUEInitialized: onIcueInitialized,
};

function onIcueInitialized() {
  // Called once when iCUE API is ready
}

function onIcueDataUpdated() {
  // Called on property change; properties available as global variables
}

// Handle late script loading
if (iCUE_initialized) {
  onIcueInitialized();
  onIcueDataUpdated();
}
```

Event	Description
<code>onICUEInitialized</code>	Called once when iCUE API and all data are ready. The event comes once when iCUE is ready.
<code>onDataUpdated</code>	Called when any meta parameter value changes. The event occurs every time any parameter changes.

Plugin Events

Each loaded plugin provides its own event object. Register plugin callbacks using `plugin{ModuleName}Events` where `{ModuleName}` is the plugin's module name with namespace removed.

Example:

```
pluginSensorsdataprovderEvents = {  
    onInitialized: onSensorPluginReady,  
};  
  
function onSensorPluginReady() {  
    window.plugins.Sensorsdataprovder.sensorValueChanged.connect(function (sensorId,  
value) {  
        // Handle sensor value change  
    });  
}  
  
if (pluginSensorsdataprovder_initialized) {  
    onSensorPluginReady();  
}
```

INFO

For detailed plugin documentation, see [Plugins](#).